

TrustedAI Movie Discovery Assistant

AI Engineer Test Report

Candidate: Nguyen Hoang Huy | **Project:** cine-reason-assistant

GitHub: github.com/nguyenhoanghuy34/cine-reason-assistant

Problem Analysis

The goal of this project is to build an AI assistant that helps users discover movies by investigating a movie dataset on their behalf. The assistant is not intended to behave like a keyword search engine. Instead, it should reason about the user's intent, retrieve the right evidence, combine multiple data signals, and explain recommendations using historical data.

Users and Needs

- The primary users are movie viewers who want personalized recommendations, explanations, and dataset-grounded answers.
- They need the assistant to understand their user ID, use their rating history, identify their preferences, and recommend movies they have not already watched.
- They also need transparent reasoning: when the assistant recommends a movie, it should show evidence such as previous high ratings, preferred genres, similar users, plot summaries, or rating statistics.

What Makes a Good Conversational Recommendation

- It should be personalized to the current user, not based only on global popularity.
- It should respect explicit constraints, such as excluding a genre or using a seed movie.
- It should be grounded in dataset evidence rather than unsupported LLM knowledge.
- It should be concise and useful: the user should see both the answer and the evidence behind it.

Key Technical Challenges

- **Agent-based routing:** a single intelligent agent autonomously determines the appropriate actions, tools, and evidence sources required to answer each user query.
- **Conversation memory:** the assistant maintains and recalls chat history within a session, allowing it to understand follow-up questions and preserve conversational context.
- **Multi-source reasoning:** the assistant must retrieve, combine, and reason over multiple data sources, such as user profiles, rating history, movie metadata, plot summaries, and similar-user information.
- **Response-time optimization:** the system must minimize unnecessary data retrieval and LLM calls while maintaining sufficient evidence and reasoning quality for fast responses.
- **System evaluation:** the assistant must be evaluated on both evidence retrieval quality and recommendation quality to identify where the system performs well and where it fails.

Approach

I decomposed the assistant into a small evidence-driven pipeline. First, an intent router identifies what the user is asking for. Then specialized tools retrieve evidence from the dataset. Finally, the LLM receives a compact evidence package and produces a short answer with supporting evidence.

System Components

- **Intent router:** classifies queries into general movie questions, personalized questions, related-user questions, or other supported requests.
- **User profile tools:** retrieve rating history, tags, favorite genres, rare genres, and blind spots.
- **Recommendation tools:** generate candidate movies using user preferences, genre constraints, quality signals, and watched-movie exclusion.
- **Title resolution tools:** resolve exact, canonical, alias, and fuzzy movie titles before lookup.
- **Content similarity tools:** use plot text with TF-IDF cosine similarity to support content-based discovery and theme comparison.
- **Similar-user tools:** retrieve similar users and aggregate their ratings for target movies.
- **Grounded answer generation:** the LLM is instructed to answer in a compact Answer and Evidence format.

Methods and Tools

- **Collaborative filtering – Pandas and Parquet:** user rating histories are processed with Pandas and stored in Parquet files. Similar-user information is used to answer questions such as “What do people with similar taste to mine think about Pulp Fiction?”
- **Content-based retrieval – TF-IDF and cosine similarity:** movie plot summaries are represented using TF-IDF vectors and compared with cosine similarity to retrieve movies with semantically related content.
- **Genre profiling – Pandas:** historical ratings are grouped and analyzed by genre to identify preferred genres, low-rated genres, and potential preference blind spots for each user.
- **Fuzzy title matching – similarity scoring:** movie titles are normalized and compared using fuzzy string similarity, allowing the system to handle alternative title formats and spelling mistakes while rejecting low-confidence matches.
- **Agent orchestration – LangChain/LangGraph:** an AI agent determines which tools and evidence sources are required for each query and coordinates the retrieval and reasoning process.
- **LLM reasoning – Groq API:** the language model receives retrieved evidence and conversation context to generate concise, dataset-grounded answers and recommendation explanations.
- **Parallel execution and token optimization:** independent evidence-retrieval operations are executed in parallel where possible, while chat history, user profiles, and retrieved evidence are compacted before being sent to the LLM to reduce latency and token usage.

Alternatives Considered

- I considered using a **vector database** for semantic retrieval, but the embedding and retrieval process introduced significant matching errors and made it more difficult to guarantee that the retrieved evidence was correctly grounded in the dataset.
- I considered building the assistant using only **LangChain**, but LangChain alone was not sufficient for an assistant that needs to coordinate multiple tools, retrieve information from different data sources, and perform multi-step reasoning. Therefore, a more structured agent-based workflow was adopted.
- I considered **self-hosting the LLM**, but the deployment complexity and available hardware resources were insufficient for efficient inference. I therefore switched to the **Groq API**, which provides significantly faster inference and reduces both implementation complexity and development time.

Decision Log

Decision	Alternative Considered	Why I Chose This
Use structured dataset retrieval instead of a vector database	Use a vector database with embedding-based semantic retrieval	Embedding and retrieval introduced matching errors that made it harder to guarantee correctly grounded evidence. Structured retrieval provides more predictable and verifiable results.
Use an agent-based workflow for multi-source reasoning	Build the assistant using only LangChain chains	The assistant needs to dynamically select tools, combine multiple evidence sources, and perform multi-step reasoning. An agent-based workflow provides more flexible orchestration for these tasks.
Use the Groq API for LLM inference	Self-host an open-source LLM	Self-hosting required more complex deployment and hardware resources than were available. Groq provides fast inference through a simple API, significantly reducing development and response time.

Evaluation

I used a mixed evaluation strategy because this system has two different goals: retrieving correct evidence and producing good recommendations. Evidence retrieval can be tested deterministically against the dataset, while recommendation quality should be evaluated with ranking metrics and qualitative failure analysis.

Functional and Evidence Tests

I created an offline evaluation suite that does not call the LLM. It checks whether the system retrieves the correct evidence for representative assignment queries.

What Each Evaluation Function Checks

The evaluation script is organized into several evaluator functions. Each function calls the same retrieval tools used by the assistant, then compares the returned evidence with deterministic expectations from the dataset.

- `evaluate_functional()` checks the basic assistant capabilities: retrieving a user's highly rated movies, deriving genre preferences, generating unwatched personalized recommendations, producing different recommendations for different users, and handling sparse user histories.
- `evaluate_content_search()` is grouped under Functional tests. It checks whether plot and meta-data search can find movies by themes such as psychological thrillers, time travel, similar themes across different genres, and movies similar to a seed title like *The Godfather*.

- `evaluate_title_resolution()` checks whether user-written movie titles are resolved to the correct dataset titles. It covers exact titles, MovieLens title-order variants, typos, aliases, similar names such as *Alien* versus *Aliens*, correct plot lookup, correct genre lookup, and unknown movies.
- `evaluate_blind_spots_and_grounding()` checks whether the system can explain genre blind spots and whether answer generation is constrained by evidence. It verifies genre exposure, high-rated versus frequently watched genres, required evidence keys, title precision/recall, and no-fabrication instructions in the final answer prompt.
- `evaluate_similar_users()` checks collaborative evidence. It finds similar users, retrieves their ratings for target movies such as *Pulp Fiction* and *Inception*, computes rating count and average rating, and confirms that the system does not invent consensus when there is no evidence.
- `evaluate_multi_signal_and_memory()` checks more complex reasoning cases. It verifies that the assistant can explain a previous recommendation, combine user history with similar-user evidence, recommend outside the user’s top genres, use content similarity from liked movies, preserve a no-*Sci-Fi* constraint, and use *Toy Story* as a positive seed while excluding *Animation*.
- `evaluate_edge_cases()` checks failure handling. It covers invalid user IDs, absent movies, low-confidence fuzzy matches, already-watched recommendation candidates, sparse histories, movies with few ratings, movies without tags, missing similar-user ratings, ambiguous titles, and contradictory genre constraints.
- `evaluate_offline_metrics()` checks recommendation ranking quality separately from evidence correctness. It hides recent positive ratings for selected users, generates recommendations from the remaining data, and reports whether the hidden positives appear in the top-*K* list.
- **Functional tests:** rating history, genre preference, personalized recommendation, sparse-user handling, and content search.
- **Title-resolution tests:** canonical titles, MovieLens title-order variation, typo/fuzzy matching, aliases, ambiguous titles, and unknown movies.
- **Evidence/grounding tests:** required evidence keys, title precision/recall, numeric grounding prompts, and no-fabrication constraints.
- **Similar-user reasoning tests:** similar users, target movie ratings, rating counts, average ratings, and insufficient-evidence cases.
- **Multi-turn and multi-signal tests:** previous recommendation reference, genre constraints, similar-user evidence, and content similarity.
- **Failure/edge-case tests:** invalid user ID, sparse history, few ratings, missing tags, no similar-user ratings, low-confidence fuzzy matches, and contradictory constraints.

Evaluation Output

```

A. Functional tests: Passed 10/10
B. Title-resolution tests: Passed 9/9
C. Evidence/grounding tests: Passed 4/4
D. Similar-user reasoning tests: Passed 4/4
E. Multi-turn and multi-signal tests: Passed 7/7
F. Failure / edge-case tests: Passed 9/9
Overall evidence/functional checks: Passed 43/43

```

The following figures provide execution evidence for the evaluation test suite. They show the successful test cases across the functional, title-resolution, grounding, similar-user reasoning, multi-turn, multi-signal, and edge-case evaluation scenarios. Overall, all 43 evidence and functional checks were successfully completed.

A. Functional tests

Passed 10/10

- PASS: 1. What movies have I rated highly?
- PASS: 2. What genres do I like the most?
- PASS: 3. What should I watch tonight?
- PASS: 4. Same recommendation query for users 1, 15, and 30
- PASS: 5. Sparse-user test using user 30
- PASS: 13. Dark psychological thriller with a twist
- PASS: 14. Movie about time travel
- PASS: 15. Two movies with similar themes but different genres
- PASS: 16. Compare Blade Runner and Twelve Monkeys by plots
- PASS: 17. Similar themes to The Godfather

Figure 1: Successful evaluation test cases – A.

B. Title-resolution tests

Passed 9/9

- PASS: 6. Exact/canonical title
- PASS: 8. MovieLens title-order variation
- PASS: 9. Typo/fuzzy title
- PASS: 10. Alias title
- PASS: 11a. Similar title Alien
- PASS: 11b. Similar title Aliens
- PASS: 6. Retrieve correct plot
- PASS: 7. Genre lookup
- PASS: 12. Unknown movie

Figure 2: Successful evaluation test cases – B.

C. Evidence/grounding tests

Passed 4/4

- PASS: 32-33. Blind spot and rarely watched genres
- PASS: 34. Highly rated genres vs watch frequency
- PASS: 35-38. Evidence key and title precision/recall
- PASS: 39-41. Numeric and answer-to-evidence consistency

Figure 3: Successful evaluation test cases – C.

D. Similar-user reasoning tests

Passed 4/4

- PASS: Similar users' opinion about Pulp Fiction
- PASS: Similar users' opinion about Inception
- PASS: 20-21. Similar-user consensus recommendation with explicit stats
- PASS: 22. Insufficient similar-user evidence

Figure 4: Successful evaluation test cases – D.

E. Multi-turn and multi-signal tests

Passed 7/7

- PASS: 26-27. Explain previous recommendation
- PASS: 23. Compare Blade Runner and Twelve Monkeys using user history and similar users
- PASS: 24. Outside top two genres but highly rated by similar users
- PASS: 25. Similar themes to liked movies from rare genre
- PASS: 28. Similar users think about the first recommendation
- PASS: 29-30. Preserve no-Sci-Fi constraint across turns
- PASS: 31. Toy Story positive seed while excluding Animation

Figure 5: Successful evaluation test cases – E.

F. Failure / edge-case tests

Passed 9/9

- PASS: 42. Invalid user ID
- PASS: 44 and 48. Absent movie / low-confidence fuzzy title
- PASS: 50. Recommendation candidate already watched
- PASS: 43. Sparse history avoids overconfidence
- PASS: 45. Very few ratings acknowledged
- PASS: 46. No tags fallback to genres/plots/ratings
- PASS: 47. No similar users rated requested movie
- PASS: 49. Multiple plausible title matches asks clarification
- PASS: 51. Contradictory constraints handled gracefully

Figure 6: Successful evaluation test cases – F.

G. Offline recommendation-quality metrics

Method: per-user hold-out. Relevant items are held-out ratings ≥ 4.0 . This evaluates retrieval quality separately from evidence grounding.

Figure 7: Successful evaluation test cases – G.

Offline Recommendation Metrics

I also ran a separate offline ranking check to distinguish recommendation quality from evidence correctness. The check hides recent high ratings, generates recommendations from the remaining data, and tests whether the hidden movies are recovered. The sampled ranking results were weak, so I do not use them as a success claim. Instead, they are reported as a limitation in the failure analysis: the assistant is currently strong at grounded evidence retrieval, but the ranking heuristic still needs a stronger recommender model.

Success Cases

Personal Rating History

- **Query:** “What movies have I rated highly?”
- **Behavior:** the assistant retrieves the current user’s ratings from `ratings.csv`.
- **Why it works:** the tool filters by the current `userId`, sorts by rating and timestamp, and joins movie metadata from `movies.csv`.

```
Ask a movie question; type /exit to quit.
User 15 | Session: 34
You: What movies have I rated highly

=====
ASSISTANT
-----
Answer:
- Bicentennial Man (1999), Limitless (2011), Aliens (1986), Alien (1979),
Star Wars: Episode VI – Return of the Jedi (1983),
Star Wars: Episode V – The Empire Strikes Back (1980),
Back to the Future Part II (1989), Terminator 2: Judgment Day (1991)
- Star Wars: Episode IV – A New Hope, The Shawshank Redemption, Schindler’s List,
Forrest Gump, Catch Me If You Can
Evidence:
- user_rating_history.top Rated movies lists eight 5-star titles (Bicentennial

EVIDENCE
-----
- intent=PERSONAL
- user_id=15
- user_top_ratings=Bicentennial Man (1999) (5.0); Limitless (2011) (5.0); Aliens (1986) (5.0)
=====
```

Figure 8: Successful retrieval of the user’s personal rating history.

Movie Title Resolution

- **Query:** “What is The Shawshank Redemption about?”
- **Behavior:** the system resolves the query to Shawshank Redemption, The.
- **Why it works:** the title resolver handles MovieLens title-order variations and retrieves the plot from `movies_with_plots.csv`.

```

You: what is The Shawshank Redemption about?

=====
ASSISTANT
-----
Answer: The Shawshank Redemption follows banker Andy Dufresne, who is wrongly convicted
of murdering his wife and her lover and sentenced to life in Shawshank State
Penitentiary. While there, he befriends fellow inmate "Red," endures brutal prison life,
uses his financial expertise to aid the warden and guards, and ultimately engineers a
daring escape.
Evidence:
- movie_summary.plot describes Andy's 1947 conviction, his friendship with Red, his work
in the prison library and finances, and his eventual escape plan.
- movie_summary.genres lists "Crime|Drama," confirming the film's focus on crime,
imprisonment, and personal redemption.
- title_resolution.match_type is "exact" for "The Shawshank Redemption," confirming the
summary applies to the requested title.

EVIDENCE
-----
- intent=GENERAL
- user_id=15
- movie_summary=Shawshank Redemption, The
=====

```

Figure 9: Successful movie title resolution and plot retrieval.

Similar-User Reasoning

- **Query:** "What do people with similar taste to mine think about Pulp Fiction?"
- **Behavior:** the system finds similar users, checks their ratings for the target movie, and aggregates rating count and average rating.
- **Why it works:** the answer is grounded in `user_similarity.parquet` and `ratings.csv`, not only LLM knowledge.

```

You: What do people with similar taste to mine think about Pulp Fiction?

=====
ASSISTANT
-----
Answer: Users who share a similar taste to you (IDs 45, 305, 380, 391, 414, 555) all
rated Pulp Fiction very positively - five gave it 5 stars and one gave it 4 stars,
yielding an average of about 4.8 / 5.
Evidence:
- movie_opinions shows ratings from similar users: 45 → 5.0, 305 → 5.0, 380 → 5.0,
391 → 5.0, 414 → 5.0, 555 → 4.0.
- similarity_ranking lists these user IDs as the users

EVIDENCE
-----
- intent=RELATED_USERS
- user_id=15
- Pulp Fiction: user 45 rated 5.0; user 160 rated 5.0; user 305 rated 5.0
- similar_users=305, 380, 391, 414, 555
- similar_users_top_movies=Star Wars: Episode IV - A New Hope (1977) (5.0); Terminator 2: Judgmen
t Day (1991) (4.89); Star Wars: Episode V - The Empire Strikes Back (1980) (4.89)
=====

```

Figure 10: Successful similar-user reasoning using rating and similarity evidence.

Failure Analysis

Failure 1: Weak Offline Ranking Quality

- **User asked:** “What should I watch tonight?”
- **System behavior:** the system generated reasonable evidence-backed recommendations, but the strict hold-out ranking check did not recover the hidden positive items for the sampled users.
- **Why it failed:** the recommender uses simple genre, rating, and popularity signals. It is explainable, but it is not trained or tuned as a ranking model.
- **What would fix it:** train a stronger hybrid recommender using collaborative filtering, matrix factorization, or learning-to-rank, then evaluate against held-out positives.

Failure 2: Sparse User History

- **User asked:** “What should I watch tonight?” for user 30.
- **System behavior:** the system can still return candidates, but the evidence is weaker because the user has limited rating history.
- **Why it failed:** sparse histories provide fewer signals for genre preference, similar-user matching, and confidence.
- **What would fix it:** ask onboarding questions, use more content-based retrieval, or incorporate stronger global priors while clearly marking uncertainty.

Failure 3: Dataset Coverage Limitations

- **User asked:** about a movie absent from the filtered dataset.
- **System behavior:** the system should not fabricate dataset evidence and should state that the movie is unavailable.
- **Why it failed or becomes limited:** the dataset is filtered and does not contain every well-known movie.
- **What would fix it:** add an external metadata source or a larger movie database, while keeping dataset-backed evidence clearly separated from external knowledge.

Failure 4: Increased Latency with Multi-Workflow Agent Design

- **Scenario:** complex queries require the agent to execute multiple reasoning and retrieval workflows before producing a final answer.
- **System behavior:** response latency increases quickly as more workflow steps, tool calls, and LLM reasoning stages are introduced.
- **Why it becomes limited:** each additional workflow may require another tool execution, data retrieval operation, or LLM call. Sequential dependencies between these operations further increase the total response time.
- **What would fix it:** reduce unnecessary workflow steps, execute independent tool calls in parallel, cache frequently used evidence, compact intermediate results, and allow the agent to select only the minimum set of tools required for each query.

Failure 5: Multi-File Retrieval and Uneven Data Distribution

- **Scenario:** some queries require evidence from multiple files such as `ratings.csv`, `movies.csv`, `movies_with_plots.csv`, and `user_similarity.parquet`.
- **System behavior:** tools can retrieve and combine these sources, but multi-file queries become more complex and the amount of available evidence varies significantly across users and movies.
- **Why it becomes limited:** the files contain different entities and levels of coverage. Some users have

many ratings while others have very few, and some movies have rich rating or tag evidence while others have limited observations. This uneven distribution makes consistent retrieval and reasoning more difficult.

- **What would fix it:** create a more unified retrieval layer with standardized identifiers and precomputed joins, improve indexing and caching, and introduce confidence or minimum-evidence thresholds so that the system can adapt its reasoning when data is sparse.

Reflection

What Works Well

- The strongest part of the system is deterministic evidence retrieval. The evaluation suite passed all functional, title-resolution, grounding, similar-user, multi-signal, and edge-case checks.
- The assistant can answer with multiple evidence sources, including rating history, genre profiles, movie metadata, plot summaries, tags, and similar-user statistics.
- Title resolution is reliable for common user input patterns, including canonical names, MovieLens title-order variants, aliases, typos, unknown titles, and near-duplicate titles such as *Alien* and *Aliens*.
- The system handles weak or missing evidence conservatively. It rejects low-confidence title matches, avoids recommending watched movies, and does not invent similar-user consensus when no ratings are available.
- The evaluation design separates evidence correctness from ranking quality, which makes the report more honest and easier to debug.

What Does Not Work Well

- The recommender is still a heuristic ranking system. It can produce grounded suggestions, but it is not yet optimized to recover hidden positives in a strict offline ranking check.
- Sparse users and low-evidence movies remain difficult because there may be too few ratings, tags, or similar-user overlaps to support confident personalization.
- Complex multi-turn requests can increase latency because the agent may need several retrieval steps before it has enough evidence to answer.
- Content similarity currently uses TF-IDF over plot text. This is deterministic and easy to evaluate, but it can miss abstract semantic similarity that embedding-based retrieval might capture better.
- Conversation state is partially dependent on chat history and LLM reference resolution, so follow-up handling would benefit from a more explicit state-tracking layer.

What I Would Improve With More Time

- Replace the current heuristic recommender with a stronger hybrid model that combines collaborative filtering, content similarity, genre preference, calibrated popularity, and recency-aware validation.
- Add a deterministic conversation-state module to track previous recommendations, rejected genres, positive seed movies, and user constraints across turns.
- Improve retrieval performance by caching frequently used evidence, precomputing common joins, and parallelizing independent tool calls.
- Compare TF-IDF plot similarity with semantic embeddings, then keep whichever method gives better grounded retrieval without introducing noisy matches.
- Add an answer-auditing layer that extracts movie titles and numeric claims from generated text and verifies them against retrieved evidence before returning the response.

Open Section

Design Principle

- The LLM is not treated as the source of truth. Dataset tools retrieve factual evidence first, and the LLM is used mainly to synthesize that evidence into a concise answer.
- This design makes the assistant easier to debug because most important claims can be traced back to ratings, movie metadata, plot summaries, tags, or similar-user statistics.

What the Evaluation Shows

- The strongest result is evidence correctness. The system passed all functional, title-resolution, grounding, similar-user, multi-signal, and edge-case checks.
- The assistant can resolve movie titles, retrieve user-specific evidence, combine multiple signals, and avoid fabricating answers when evidence is missing or weak.

Main Limitation

- The main limitation is recommendation ranking quality. The current recommender is explainable and grounded, but it is still based on heuristic signals rather than a trained ranking model.
- Because of that, the assistant can justify why a recommendation is reasonable, but it should not be presented as a high-performance recommendation engine yet.

Next Direction

- The next improvement would be to keep the evidence-grounded architecture while replacing the heuristic ranking layer with a stronger hybrid recommender.
- A better version would combine collaborative filtering, content similarity, calibrated popularity, explicit constraints, and validation-driven tuning.